Breadth-First & Depth-First Search

A practice workbook — worked examples, 12 exercises, full answer key

How to use this workbook

Work through the two worked traces first, then attempt the exercises without looking ahead. The answer key starts on the page marked **Answer Key** — check only after you have committed to an answer. Every traversal here follows one convention, and you should follow it too:

> **Tie-breaking rule:** whenever a vertex has several unvisited neighbours, visit them in **alphabetical / numerical order**. Without a stated rule a graph has many valid BFS and DFS orders, so exams always fix one.

The two algorithms side by side

	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO), or recursion
Explores	All vertices at distance k before k+1	One path as deep as possible, then backtracks
Time	O(V + E)	O(V + E)
Space	O(V) — worst case the whole frontier	O(V) — worst case the recursion depth
Shortest path?	Yes, on unweighted graphs	No — finds *a* path, not the shortest
Typical uses	Shortest hops, level order, bipartite check	Cycle detection, topological sort, components

Pseudocode

BFS

```
BFS(G, s):
  for each v: visited[v] = false
  visited[s] = true; dist[s] = 0
  Q = empty queue; Q.enqueue(s)
  while Q not empty:
      u = Q.dequeue()
      visit(u)
      for each neighbour v of u:
          if not visited[v]:
              visited[v] = true
              dist[v]   = dist[u] + 1
              parent[v] = u
              Q.enqueue(v)
```

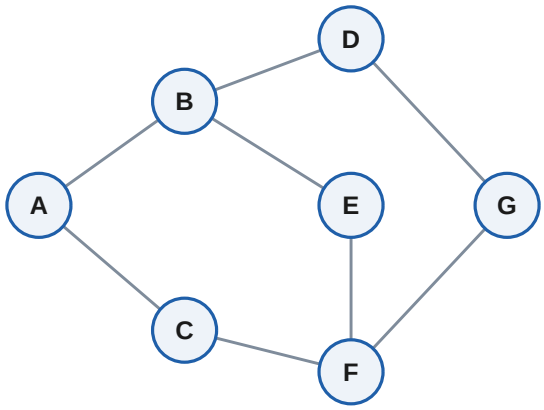
DFS

```
DFS(G, u):            # recursive
  visited[u] = true
  visit(u)
  for each neighbour v of u:
      if not visited[v]:
          DFS(G, v)
DFS-ITER(G, s):       # explicit stack
  St = empty stack; St.push(s)
  while St not empty:
      u = St.pop()
      if visited[u]: continue
      visited[u] = true; visit(u)
      for each neighbour v of u
          (in reverse order):
          if not visited[v]: St.push(v)
```

Watch out: mark a vertex as visited when you *enqueue* it in BFS (not when you dequeue it), otherwise a vertex can enter the queue several times. In the iterative DFS the check happens on *pop*, which is why a vertex may sit on the stack more than once — that is fine, the *visited* test discards the duplicates.

Worked example

Undirected graph **G**, traversals start at **A**, neighbours taken alphabetically.



BFS from A — step by step

Step	Dequeued	Newly enqueued	Queue after step
1	A	B, C	B C
2	B	D, E	C D E
3	C	F	D E F
4	D	G	E F G
5	E	—	F G
6	F	—	G
7	G	—	(empty)

BFS visit order: A → B → C → D → E → F → G

Distances from A (in edges): A=0, B=1, C=1, D=2, E=2, F=2, G=3

Read it as levels: level 0 is {A}, level 1 is {B, C}, level 2 is {D, E, F}, level 3 is {G}. BFS never touches a level until the previous one is exhausted — that is exactly why the first time it reaches a vertex it has used the fewest possible edges.

DFS from A — step by step

Go as deep as possible before backtracking. From A take B (alphabetically first), from B take D, from D take G, from G take F, from F take C; C's only unvisited option is none, so back up to F, which still has E. That gives:

DFS visit order: A → B → D → G → F → C → E

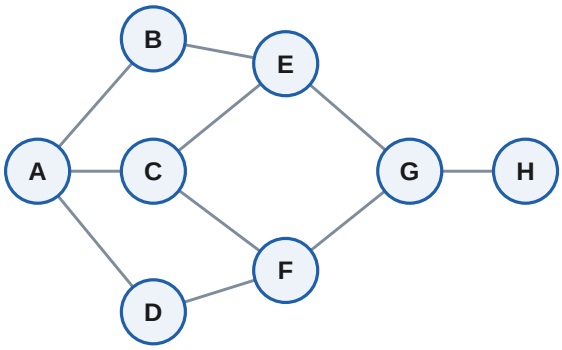
Same result with an explicit stack: A → B → D → G → F → C → E

> **Sanity check you can always run:** both traversals must list every vertex reachable from the start exactly once. If a vertex appears twice, you forgot to mark it visited; if one is missing, it sits in a different connected component.

Exercises

The answer key starts on page 7. Neighbours in alphabetical / numerical order throughout.

Question 1 — BFS order and distances



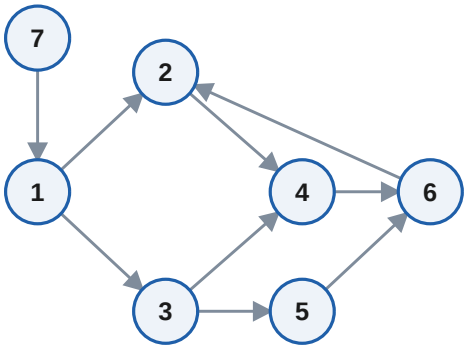
Graph **G1** (undirected). Starting at **A**:

(a) Give the BFS visit order.
(b) Give the distance from A to every vertex.
(c) Which vertices are at level 2?

Question 2 — DFS on the same graph

Using **G1** above and starting at **A**: (a) give the recursive DFS visit order; (b) give the order produced by the iterative stack version that pushes neighbours in reverse alphabetical order; (c) does DFS find the shortest A–H path here?

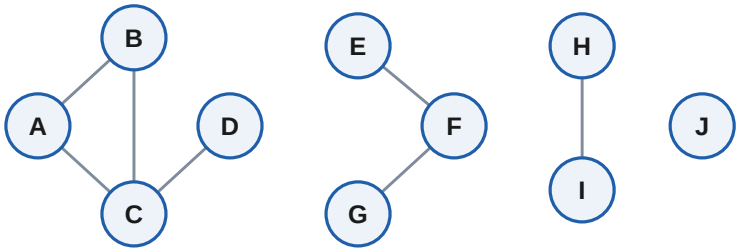
Question 3 — Directed graph



Graph **G2** (directed). Starting at **1**:

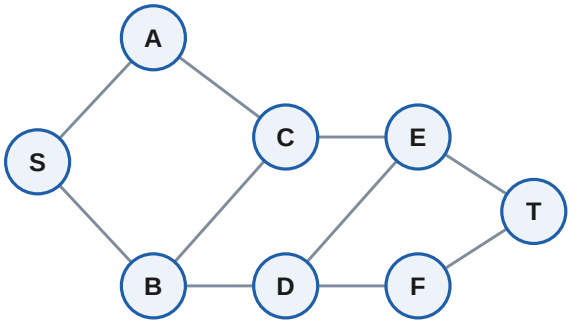
(a) BFS order.
(b) DFS order.
(c) Which vertices are unreachable from 1, and why does that not mean they are disconnected?

Question 4 — Connected components



Graph **G3** (undirected; vertex J has no edges). (a) How many connected components are there? (b) List the vertices of each. (c) Sketch the loop you would wrap around DFS to find components in a graph where you are *not* told the components in advance.

Question 5 — Shortest path reconstruction

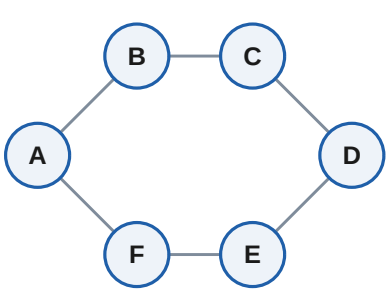


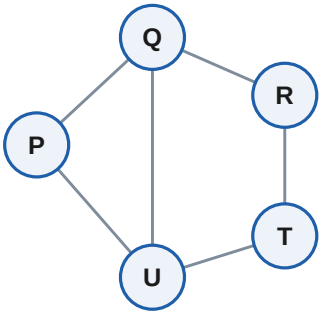
Graph **G4** (undirected). Run BFS from **S**.

(a) What is the shortest distance from S to T?
(b) Fill in the parent array, then reconstruct the actual path S…T.
(c) Is your path the only shortest one?

Question 6 — Bipartite check by 2-colouring

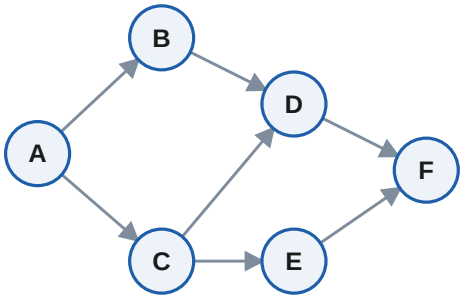
Run BFS from the leftmost vertex of each graph, colouring each newly discovered vertex the opposite colour to its parent. One of these graphs is bipartite and one is not — say which, and name the edge that proves it.





Left: **G5a** (start at A) | Right: **G5b** (start at P)

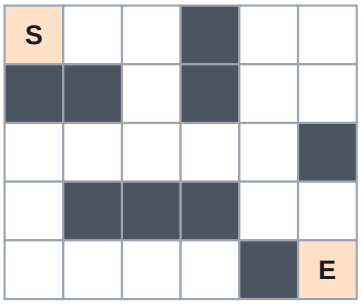
Question 7 — Discovery / finish times and topological order



DAG **G6**. Run DFS from **A**, using a single counter that ticks once on discovery and once on finish.

(a) Give *d*[v] and *f*[v] for every vertex.
(b) Sort vertices by decreasing finish time — this is a topological order. Write it out.
(c) Verify it: does every edge point forward in your list?

Question 8 — BFS on a grid (shortest path in a maze)



Move up / down / left / right only; # cells are walls. Treat every open cell as a vertex and every legal move as an edge of weight 1.

(a) What is the fewest number of moves from **S** to **E**?
(b) Write one shortest path as a list of moves.
(c) Why would DFS be a poor choice here?

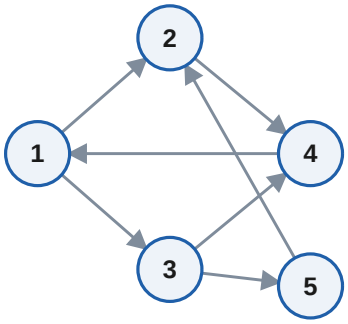
Question 9 — From an adjacency list (no picture)

```
A: B, F
B: A, C, G
C: B, D, G
D: C, E
E: D, F
F: A, E, G
G: B, C, F
```

Graph **G8** given only as an adjacency list. Starting at **A**:

(a) BFS order.
(b) DFS order.
(c) Draw the graph, then check whether it contains a cycle.

Question 10 — Edge classification



Directed graph **G7**. Run DFS from **1** and classify every edge as **tree**, **back**, **forward** or **cross**.

Reminder: a *back* edge points to a vertex that is still on the recursion stack — and its existence is exactly what proves the graph has a cycle.

(a) Classify all 7 edges. (b) Is G7 acyclic?

Question 11 — Short answer

(a) A graph has 10^6 vertices but the target is only 3 edges away from the start. Which traversal do you run, and what goes wrong with the other one?

(b) BFS gives shortest paths on unweighted graphs. Give a concrete weighted example where BFS returns a path that is *not* cheapest, and name the algorithm you should use instead.

(c) You run DFS recursively on a path graph of 10^5 vertices and the program crashes. What happened, and what is the fix?

(d) Both BFS and DFS run in O(V + E). Why is it O(V + E) and not O(V × E)?

Question 12 — Trace table to complete

Fill in this BFS trace for graph **G4** (Question 5) starting at **S**. Add rows as needed.

Step	Dequeued	Newly enqueued	Queue after step
1			
2			
3			
4			
5			
6			
7			
8			

Answer Key

Question 1

(a) BFS order: **A** → **B** → **C** → **D** → **E** → **F** → **G** → **H**

(b) Distances from A: A=0, B=1, C=1, D=1, E=2, F=2, G=3, H=4

(c) Level 2 = {E, F}. These are exactly the vertices first reached on the second sweep of the queue.

Question 2

(a) Recursive DFS: **A** → **B** → **E** → **C** → **F** → **D** → **G** → **H**

(b) Iterative DFS pushing in reverse alphabetical order: **A** → **B** → **E** → **C** → **F** → **D** → **G** → **H** — identical here, which is the point of reversing the push order.

(c) No. DFS reaches H, but only BFS guarantees the shortest route. The true shortest A–H path is A → B → E → G → H, of length 4.

Question 3

(a) BFS from 1: **1** → **2** → **3** → **4** → **5** → **6**

(b) DFS from 1: **1** → **2** → **4** → **6** → **3** → **5**

(c) Unreachable from 1: {7}. Vertex 7 has an edge *into* 1 but none out of 1 reaches it. In a directed graph reachability is one-way, so the underlying undirected graph can still be perfectly connected — which it is here.

Question 4

(a) **4** connected components.

(b) {A, B, C, D} | {E, F, G} | {H, I} | {J}

(c) Loop over every vertex; if it is still unvisited, start a fresh DFS (or BFS) from it and label everything that search reaches with the current component number, then increment the counter. Total cost is still O(V + E) because each vertex and edge is touched once overall.

Question 5

(a) Shortest distance S to T = **4** edges.

(b) Parent array: A←S, B←S, C←A, D←B, E←C, F←D, T←E. Following parents back from T: **S** → **A** → **C** → **E** → **T**.

(c) No — S→B→D→E→T and S→B→D→F→T are also length 4. BFS returns one shortest path, the one determined by your tie-breaking rule, not all of them.

Question 6

G5a is bipartite. The 2-colouring splits it as {A, C, E} and {B, D, F}; every edge runs between the two sets. It is a 6-cycle, and all even cycles are bipartite.

G5b is not bipartite. BFS colouring hits the edge **Q–U**, whose endpoints already carry the same colour. That conflict exposes the triangle **P–Q–U–P**, an odd cycle of length 3 (the 5-cycle P–Q–R–T–U–P is another). A graph is bipartite if and only if it contains no odd cycle.

Question 7

(a) A: d=1 f=12, B: d=2 f=7, C: d=8 f=11, D: d=3 f=6, E: d=9 f=10, F: d=4 f=5

(b) Decreasing finish time gives the topological order: **A** → **C** → **E** → **B** → **D** → **F**

(c) Every edge of G6 goes left-to-right in that list, so the order is valid. Note that a DAG usually has several valid topological orders; DFS finish times give you one of them.

Question 8

(a) Fewest moves = **9**.

(b) One shortest path (cells as row,col with the top-left at 0,0): (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (3,5) → (4,5).
Moves: Right, Right, Down, Down, Right, Right, Down, Right, Down.

(c) DFS would happily wander down a long dead-end corridor and report the first route it stumbles on, which can be far longer than 9. On an unweighted grid BFS is the standard choice; each cell is dequeued once, so it is O(rows × cols).

Question 9

(a) BFS from A: **A** → **B** → **F** → **C** → **G** → **E** → **D**

(b) DFS from A: **A** → **B** → **C** → **D** → **E** → **F** → **G**

(c) Yes, it has cycles — for example A–B–G–F–A. Quick test: an undirected graph with V vertices and more than V–1 edges must contain a cycle. Here V=7 and E=9.

Question 10

(a) Discovery/finish: 1: d=1 f=10, 2: d=2 f=5, 3: d=6 f=9, 4: d=3 f=4, 5: d=7 f=8

From	To	Edge type
1	2	tree
2	4	tree
4	1	back
1	3	tree
3	4	cross
3	5	tree
5	2	cross

(b) The graph contains a back edge, so it is **cyclic** — the cycle is 1 → 2 → 4 → 1.

Question 11

(a) **BFS.** It stops after expanding three levels, having touched only the vertices within distance 3. DFS could plunge millions of vertices deep along the wrong branch before ever checking a vertex that was three steps away.

(b) Take S→A with weight 100 and S→B→A with weights 1 and 1. BFS returns S→A because it is one edge, cost 100; the cheaper route costs 2. Use **Dijkstra** for non-negative weights (or Bellman–Ford if negative weights exist).

(c) **Stack overflow** — recursion depth grows to 10^5 frames. Rewrite DFS iteratively with an explicit stack, or raise the interpreter's recursion limit (in Python, `sys.setrecursionlimit`) as a stopgap.

(d) Each vertex is dequeued/popped exactly once, costing O(V) overall. When a vertex is processed you scan its adjacency list once; summed over all vertices that is O(E) for a directed graph, O(2E) for undirected. The two costs add, they do not multiply — you never re-scan a vertex's list.

Question 12

Completed BFS trace for G4 from S:

Step	Dequeued	Newly enqueued	Queue after step
1	S	A, B	A B
2	A	C	B C
3	B	D	C D
4	C	E	D E

5	D	F	E F
6	E	T	F T
7	F	—	T
8	T	—	(empty)

Final BFS order: **S** → **A** → **B** → **C** → **D** → **E** → **F** → **T**

> **If you want more reps:** take any graph in this workbook, change the start vertex, and redo both traversals — the orders change but the set of reached vertices does not. Then reverse the tie-breaking rule to descending order and watch how DFS changes far more dramatically than BFS.